

Student

Chapter 1 covered the fact table side in depth. Now I want the same precision on the dimension side - the patterns that go beyond a simple flat lookup table.

Claude • AI Assistant

Most real warehouses quickly outgrow simple flat dimensions. A single Date dimension serves as order date, ship date, and delivery date simultaneously.

A patient can have multiple diagnoses. A product hierarchy can have uneven depth.

Attributes like customer age or income bracket change so frequently that Type 2 would generate millions of new rows. Each of these situations has a canonical dimensional modeling solution - and each is a standard interview topic.

Five precision subsections:

1. **Role-playing dimensions** - one physical table, multiple meanings in the same fact query.
2. **Bridge tables** - handling many-to-many relationships between facts and dimensions.
3. **Hierarchies in dimensions** - fixed, ragged, and recursive structures for drill-down.
4. **Date and time dimensions** - the most reused dimension in any warehouse, built correctly.
5. **Mini-dimensions** - splitting rapid-change attributes out of large SCDs to prevent row explosion.

CONCEPT CARD

The dimension design principles this chapter applies:

1. **One physical table, many logical roles** - role-playing avoids redundant copies.
2. **Bridge tables absorb complexity** - many-to-many belongs in a bridge, not in the fact or dimension.
3. **Hierarchy depth belongs in the dimension** - not computed at query time via recursive SQL.
4. **Date dimension is pre-built, never computed** - `DATEPART()` in SQL is not a substitute for a full calendar dimension.
5. **Rapid-change attributes split, not Type-2'd** - mini-dimensions keep large SCDs stable.

SUMMARY CARD

By the end of this chapter you will be able to:

- Design role-playing dimensions using views and explain the query implications.
- Build a bridge table for a multi-valued dimension and write a weighted allocation query.
- Represent fixed, ragged, and recursive hierarchies in a flat dimension table.
- Build a complete date dimension with fiscal calendar support.
- Recognize when a mini-dimension is needed and design one correctly.

🌟 AI DEEP DIVE PROMPT

Warm-up: **"**A retail data warehouse has a single *dimdate* table. The *factorders* table needs to reference it three times: as *orderdate*, *shipdate*, and *delivery_date*.

Explain why this is called a role-playing dimension, show the DDL for the fact table, and write a query that uses all three date roles simultaneously."**"**

	Problem it solved	Key structural element
ROLE-PLAYING DIMENSIONS	One date dimension must serve as <i>order_date</i> , <i>ship_date</i> , <i>delivery_date</i> in the same query	Views or aliases over a single physical <i>dim_date</i> table — no data duplication
BRIDGE TABLES	A patient can have 5 diagnoses; a single FK in the fact table cannot represent all of them	Fact → bridge → dimension; optional weighting factor column for allocation
HIERARCHIES	Products roll up through subcategory → <i>category</i> → department, but org org chart levels vary	Fixed: flattened columns (<i>level1</i> , <i>level2</i> , <i>level3</i>). Ragged: <i>is_leaf</i> flag. Recursive: <i>parent_key</i> self-reference
DATE DIMENSION	DATEPART() is slow, fiscal calendars and holiday flags cannot be computed on the fly	Pre-built table with one row per day. Fiscal year, holiday flag, week-of-year, relative period columns
MINI-DIMENSIONS	Customer age, income bracket, and loyalty tier change monthly — <i>Type 2</i> would generate millions of rows	Fast-changing attributes split into a separate small dimension with its own key in the fact table

These five patterns cover 95% of dimension design challenges you will encounter in production warehouses.

Advanced Dimension Patterns — Five Techniques and Their Problems Solved.

Advanced Dimension Patterns - Five Techniques and Their Problems Solved

Five Advanced Dimension Techniques - Quick Reference			
Technique	Core mechanism	Key DDL element	Primary benefit
Role-playing dim	One physical table, N logical views	CREATE VIEW <i>orddate</i> AS SELECT * FROM <i>dimdate</i>	No data duplication; single point of maintenance
Bridge table	Fact → bridge → dimension with weight	<i>factkey</i> , <i>dimkey</i> , weight_factor DECIMAL	Correct many-to-many; enables proportional allocation

Technique	Core mechanism	Key DDL element	Primary benefit
Fixed hierarchy	Flattened level columns in dimension	<i>level1key, level2key, ... levelN_name</i>	Fast drill-down without recursive queries
Date dimension	Pre-built calendar table	One row per day; <i>fiscalyear, isholiday, relative_period</i>	Fiscal support, holiday filtering, query performance
Mini-dimension	Fast-changing attrs split to separate dim	<i>demokey FK in fact table + dimcustomer_demo</i>	Stable large SCD + affordable change tracking

sql – Skeleton DDL showing all five techniques in a single retail schema

```
-- ROLE-PLAYING: three views over one physical dim_date
CREATE VIEW dim_order_date AS SELECT * FROM dim_date;
CREATE VIEW dim_ship_date AS SELECT * FROM dim_date;
CREATE VIEW dim_delivery_date AS SELECT * FROM dim_date;

-- BRIDGE TABLE: customer account with multiple holders
CREATE TABLE bridge_account_holder (
    account_key INT NOT NULL,
    customer_key INT NOT NULL,
    weight_factor DECIMAL(5,4) NOT NULL DEFAULT 1.0,
    PRIMARY KEY (account_key, customer_key)
);

-- FIXED HIERARCHY: product rolled up four levels
CREATE TABLE dim_product (
    product_key INT PRIMARY KEY,
    product_name VARCHAR(200),
    subcategory_name VARCHAR(100),
    category_name VARCHAR(100),
    department_name VARCHAR(100)
    -- all hierarchy levels flattened into one row
);

-- DATE DIMENSION: pre-built calendar
CREATE TABLE dim_date (
    date_key INT PRIMARY KEY, -- YYYYMMDD integer
    calendar_date DATE NOT NULL,
    day_of_week VARCHAR(10),
    is_weekend TINYINT,
```

```

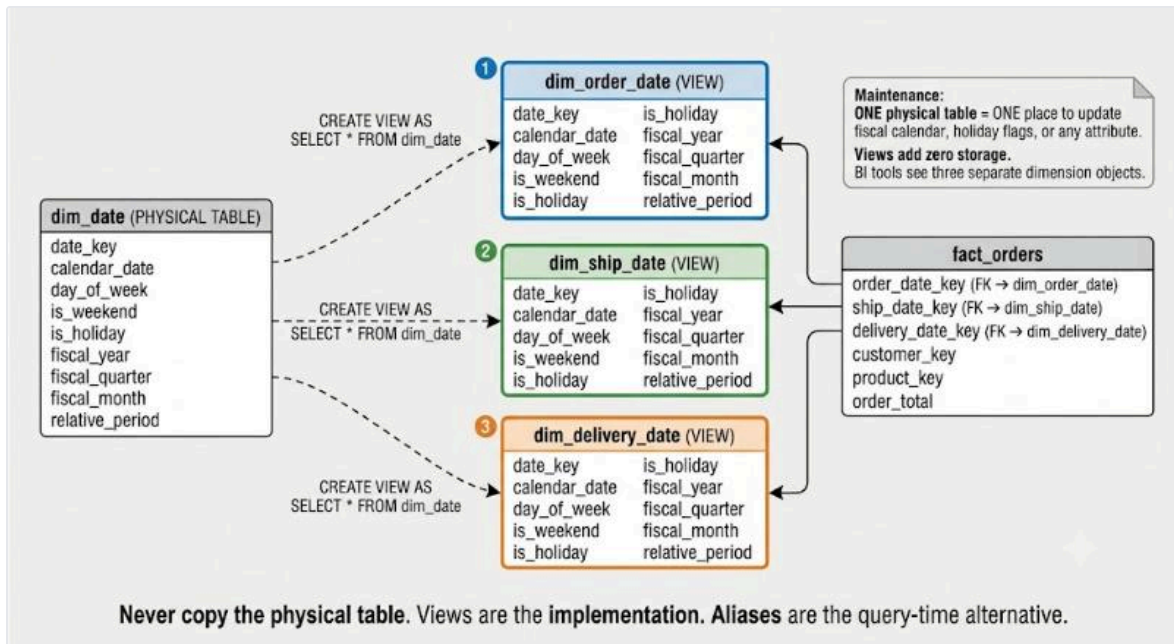
is_holiday      TINYINT,
fiscal_year      SMALLINT,
fiscal_quarter   TINYINT,
fiscal_month     TINYINT,
relative_period  INT    -- 0=current month, -1=last month, etc.
);

-- MINI-DIMENSION: rapidly changing customer demographics
CREATE TABLE dim_customer_demo (
    demo_key      INT PRIMARY KEY,
    age_band      VARCHAR(20),    -- '25-34', '35-44'
    income_band    VARCHAR(20),    -- 'Low', 'Medium', 'High'
    loyalty_tier   VARCHAR(15)     -- 'Bronze', 'Silver', 'Gold', 'Platinum'
);

```

ROLE-PLAYING DIMENSIONS — ONE TABLE, MULTIPLE MEANINGS

Claude • AI Assistant



Role-Playing Dimensions - One Physical Table, Three Logical Views

sql – Role-playing dimension – views, fact DDL, and multi-role query

```

-- =====
-- STEP 1: One physical date dimension
-- =====

CREATE TABLE dim_date (
    date_key          INT PRIMARY KEY,
    calendar_date     DATE          NOT NULL,
    day_name          VARCHAR(10) NOT NULL,
    day_of_month      TINYINT       NOT NULL,
    month_name        VARCHAR(10) NOT NULL,
    month_number      TINYINT       NOT NULL,
    quarter_number    TINYINT       NOT NULL,
    year_number       SMALLINT      NOT NULL,
    is_weekend        TINYINT       NOT NULL DEFAULT 0,
    is_holiday        TINYINT       NOT NULL DEFAULT 0,
    fiscal_year       SMALLINT,
    fiscal_quarter    TINYINT,
    fiscal_month      TINYINT
);

-- =====
-- STEP 2: Three views – one per role
-- =====

CREATE VIEW dim_order_date AS SELECT * FROM dim_date;
CREATE VIEW dim_ship_date AS SELECT * FROM dim_date;
CREATE VIEW dim_delivery_date AS SELECT * FROM dim_date;

-- =====
-- STEP 3: Fact table – one FK column per role
-- =====

CREATE TABLE fact_orders (
    order_date_key    INT          NOT NULL, -- FK → dim_order_date
    ship_date_key     INT,          -- FK → dim_ship_date
    (nullable)
    delivery_date_key INT,          -- FK → dim_delivery_date
    (nullable)
    customer_key      INT          NOT NULL,
    store_key         INT          NOT NULL,
    product_key       INT          NOT NULL,
    order_total       DECIMAL(12,2),
    items_ordered     INT
);

-- =====
-- STEP 4: Query using all three roles simultaneously
-- =====

```

```

SELECT
    ord.fiscal_year                                AS
order_fiscal_year,
    ord.month_name                                AS
order_month,
    shp.day_name                                  AS
ship_day_of_week,
    COUNT(*)                                      AS
order_count,
    SUM(CASE WHEN del.date_key IS NULL THEN 1 ELSE 0 END) AS
undelivered_count,
    AVG(del.date_key - ord.date_key)              AS
avg_days_to_deliver
FROM    fact_orders      f
JOIN    dim_order_date   ord ON f.order_date_key   = ord.date_key
LEFT JOIN dim_ship_date  shp ON f.ship_date_key    = shp.date_key
LEFT JOIN dim_delivery_date del ON f.delivery_date_key = del.date_key
WHERE   ord.fiscal_year = 2024
GROUP  BY ord.fiscal_year, ord.month_name, shp.day_name
ORDER  BY ord.fiscal_year, ord.month_name;

```

Student

My fact_orders table needs three date references - order date, ship date, delivery date. Do I need three separate date dimension tables?

Claude • AI Assistant

No - and copying the same table three times would be a serious mistake. You need one physical date dimension and three **logical roles** that each point to it.

This is the **role-playing dimension** pattern: a single physical table that participates in multiple foreign-key relationships within the same fact table, each relationship giving the dimension a different semantic meaning.

CONCEPT CARD

Why one physical table is correct:

1. **Single point of maintenance** - a correction to the holiday flag on a date row fixes it for all three roles simultaneously.
2. **No storage duplication** - a date dimension covering 10 years has ~3,650 rows. Copying it three times triples the storage for no analytical benefit.

3. **Consistent attributes** - if a fiscal year definition changes, you update one table, not three.
4. **The roles are logical** - the physical table is `dim_date` . The roles (`order_date` , `ship_date` , `delivery_date`) are implemented as database views or query-level aliases.

CONCEPT CARD

Implementation - two approaches:

1. **View approach** - create a view for each role: `CREATE VIEW dim_order_date AS SELECT * FROM dim_date` . The fact table's `order_date_key` has a documented FK to `dim_order_date` . Downstream BI tools see three separate dimension objects.
2. **Alias approach** - join the same physical table multiple times with different aliases in SQL: `JOIN dim_date ord ON f.order_date_key = ord.date_key JOIN dim_date shp ON f.ship_date_key = shp.date_key` . No DDL needed; aliasing is at query time.
3. **View is preferred in production** - BI tools (Power BI, Tableau, MicroStrategy) expect one dimension object per role. Aliasing requires custom SQL and bypasses the semantic layer.

CONCEPT CARD

Fact table design with role-playing date keys:

1. Every role gets its own foreign key column in the fact table.
2. All FKs point to the same physical `dim_date` table (or its views).
3. Milestone-date columns that haven't been reached are NULL - same as accumulating snapshot nullability.
4. Query pattern: each role's key joins to a view or alias; attributes are selected from that alias.

EXAMPLE CARD

Order fulfillment query using all three date roles:

sql - Code Example

```
SELECT
    ord.fiscal_month                AS order_month,
    shp.day_of_week                 AS ship_day,
    AVG(del.date_key - ord.date_key) AS avg_days_to_deliver,
    SUM(f.order_total)              AS revenue
FROM    fact_orders                f
JOIN    dim_date  ord ON f.order_date_key  = ord.date_key
JOIN    dim_date  shp ON f.ship_date_key   = shp.date_key
JOIN    dim_date  del ON f.delivery_date_key = del.date_key
WHERE   ord.fiscal_year = 2024
```

```
GROUP BY ord.fiscal_month, shp.day_of_week
ORDER BY ord.fiscal_month;
```

🔗 ANALOGY

A role-playing dimension is like a **Swiss Army knife** used in three ways at the same dinner table. The tool (dim_date) is one physical object.

Your right hand uses it as a corkscrew (order date), your left hand uses it as a knife (ship date), and it sits in the middle as a bottle opener (delivery date). It is one knife serving three roles simultaneously - not three separate knives.

⚠️ WARNING

Some BI tools handle role-playing dimensions poorly - they display only one relationship between a fact and a dimension even when multiple FK columns exist. In these tools, the **view approach is mandatory**: create three separate view-based dimension objects, one per role, so the BI tool sees them as independent dimensions.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Creating three physical copies of `dim_date` - `dim_order_date` , `dim_ship_date` , `dim_delivery_date` - as separate tables with identical rows. Now a fiscal calendar change requires three UPDATE statements.

A holiday correction requires three rows changed. Maintenance becomes $O(n \times \text{roles})$.

Correct Way: One physical `dim_date` . Three views. Each view is one role. One maintenance point.

EXAM TIP CARD

Interview classic: "What is a role-playing dimension and how do you implement it?" Winning answer: "A role-playing dimension is a single physical dimension table that participates in multiple foreign-key relationships in the same fact table, each with a different semantic meaning - like a date dimension serving as `order_date`, `ship_date`, and `delivery_date`.

Implementation: create a database view per role (CREATE VIEW `dim_ship_date` AS SELECT FROM `dim_date`).

The fact table holds one FK column per role. BI tools see each view as a distinct dimension object.

Never copy the physical table - one maintenance point is the entire point."

🔥 EXERCISE

Your `fact_insurance_claims` table has five date references: `reported_date_key` , `incident_date_key` , `assessment_date_key` , `payment_date_key` , `closed_date_key` . (a) Write the five CREATE VIEW statements.

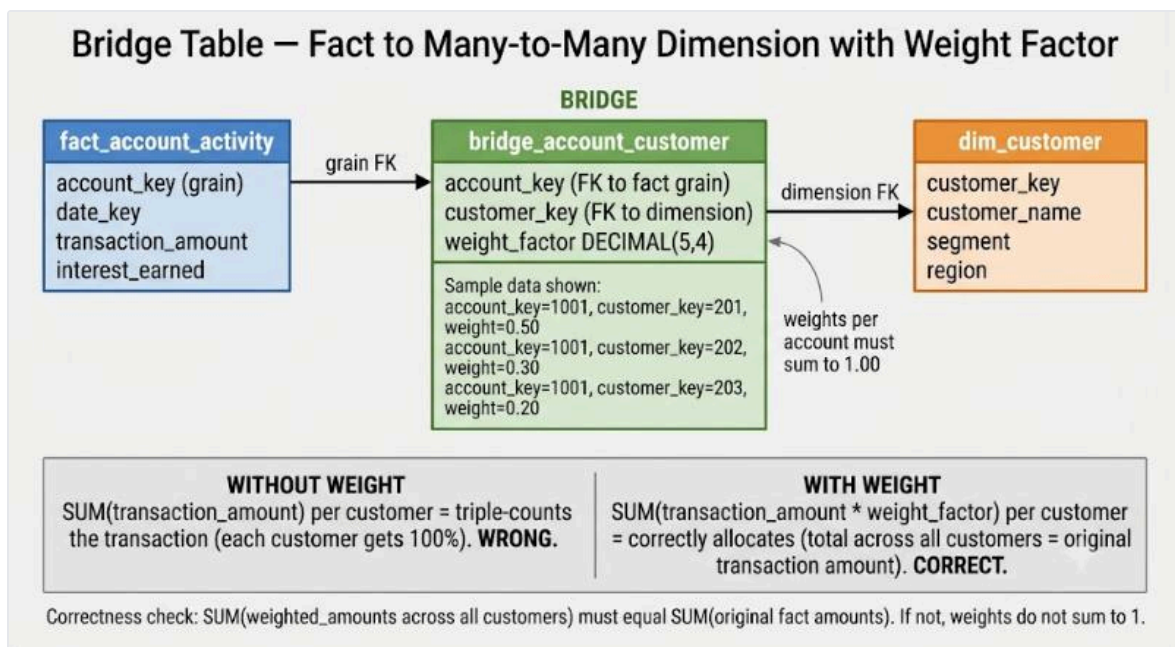
(b) Write a query calculating average days from incident to payment, broken down by month of incident and day-of-week of payment.

AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Design a role-playing dimension setup for a logistics company whose shipment fact table references: pickup_date, expected_delivery_date, actual_delivery_date, and exception_date. Include: the physical dim_date DDL, four CREATE VIEW statements, the fact table DDL with four date FKs, and a query calculating on-time delivery rate by pickup month."

BRIDGE TABLES — MANY-TO-MANY AND MULTI-VALUED DIMENSIONS

Claude • AI Assistant



Bridge Table - Fact to Many-to-Many Dimension with Weight Factor

sql – Bridge table DDL and weighted allocation query – bank account example

```

-- =====
-- BRIDGE TABLE: account → multiple customers
-- =====
CREATE TABLE bridge_account_customer (
    account_key    INT          NOT NULL, -- FK → fact grain / dim_account
    customer_key   INT          NOT NULL, -- FK → dim_customer

```

```

weight_factor DECIMAL(5,4) NOT NULL, -- must sum to 1.0 per account_key
is_primary TINYINT NOT NULL DEFAULT 0,
PRIMARY KEY (account_key, customer_key)
);

-- Sample data: account 1001 has three holders
INSERT INTO bridge_account_customer VALUES
    (1001, 201, 0.5000, 1), -- primary holder: 50%
    (1001, 202, 0.3000, 0), -- secondary: 30%
    (1001, 203, 0.2000, 0); -- tertiary: 20%
-- weights: 0.5 + 0.3 + 0.2 = 1.0 ✓

-- =====
-- WEIGHTED ALLOCATION QUERY
-- Business: revenue attributed to each customer segment
-- =====
SELECT
    c.customer_segment,
    SUM(f.transaction_amount * b.weight_factor) AS attributed_revenue,
    COUNT(DISTINCT f.account_key) AS account_count
FROM fact_account_activity f
JOIN bridge_account_customer b ON f.account_key = b.account_key
JOIN dim_customer c ON b.customer_key = c.customer_key
JOIN dim_date d ON f.date_key = d.date_key
WHERE d.fiscal_year = 2024
GROUP BY c.customer_segment
ORDER BY attributed_revenue DESC;

-- =====
-- CORRECTNESS CHECK: weighted totals must equal fact totals
-- =====
SELECT
    SUM(f.transaction_amount) AS fact_total,
    SUM(f.transaction_amount * b.weight_factor) AS weighted_total,
    ABS(SUM(f.transaction_amount)
        - SUM(f.transaction_amount * b.weight_factor)) AS discrepancy
FROM fact_account_activity f
JOIN bridge_account_customer b ON f.account_key = b.account_key;
-- discrepancy should be 0 (or < 0.01 for rounding)

-- =====
-- FILTER BY DIMENSION ATTRIBUTE THROUGH BRIDGE
-- Business: all accounts linked to customers in 'Gold' segment
-- =====
SELECT DISTINCT f.account_key
FROM fact_account_activity f

```

```
JOIN    bridge_account_customer b ON f.account_key = b.account_key
JOIN    dim_customer           c ON b.customer_key = c.customer_key
WHERE   c.loyalty_tier = 'Gold';
```

Student

What happens when the relationship between a fact and a dimension is many-to-many? For example, one bank account can have multiple customers, and one customer can have multiple accounts.

Claude • AI Assistant

This is the **multi-valued dimension problem** - one of the most important advanced patterns in Kimball's methodology. A single foreign key in the fact table can represent only one dimension member per row.

When the relationship is genuinely many-to-many, you need a **bridge table** sitting between the fact and the dimension to absorb the multiplicity.

CONCEPT CARD

The multi-valued dimension problem:

1. **Single FK fails** - `fact_account_activity` has `customer_key INT`. Account #1001 has three holders. Which `customer_key` do you store? Picking one loses two.
2. **Repeating rows in fact fails** - three rows for account #1001, one per customer. Now `SUM(balance)` triple-counts the account balance. Aggregation is broken.
3. **Multi-value column fails** - `customer_keys = '101,102,103'`. Not relational; not joinable; not filterable.
4. **Bridge table is the solution** - `bridge_account_customer(account_key, customer_key, weight_factor)` correctly represents all three holders with one row each.

CONCEPT CARD

Bridge table structure:

1. **Columns** - at minimum: the fact's grain key + the dimension key. Optionally: `weight_factor DECIMAL(5,4)`.
2. **Weight factor** - allocates the fact's measure across dimension members. Three equal holders: weight = 0.333. Primary account holder: weight = 0.6; secondary: 0.4. Weights must sum to 1.0 per fact row.

3. **No measures** - the bridge table holds only keys and the optional weight. All measures stay in the fact.
4. **Query pattern** - fact JOIN bridge ON fact.grainkey = bridge.grainkey, then bridge JOIN dimension ON bridge.dimkey = dimension.dimkey. Filter by dimension attributes after the bridge join.

CONCEPT CARD

Weighted allocation query:

1. **The business question** - 'What is the revenue attributable to each customer, across all accounts they hold?'
2. **Without weight** - each customer gets 100% of the account's revenue. Multi-counting.
3. **With weight** - `SUM(f.revenue * b.weight_factor)` - each customer gets their proportional share. Totals foot correctly to the fact total.
4. **Rule** - if weights sum to 1.0 per account, `SUM(weighted_revenue)` across all customers equals `SUM(revenue)` in the fact. This is the correctness check.

EXAMPLE CARD

Healthcare: patient with multiple diagnoses:

1. `fact_hospital_encounter(encounter_key, patient_key, date_key, total_cost)` - one row per visit.
2. `dim_diagnosis(diagnosis_key, icd_code, diagnosis_name, category)` - each ICD code.
3. `bridge_encounter_diagnosis(encounter_key, diagnosis_key, is_primary TINYINT)` - one row per diagnosis per encounter.
4. Query: 'Total cost attributed to diabetes encounters in 2024' → JOIN via bridge, filter `dim_diagnosis.category = 'Diabetes'`.
5. No weight needed here - cost goes to the encounter, not split by diagnosis. Weight is needed when allocating revenue across account holders.

🔗 ANALOGY

A bridge table is like a **many-to-many relationship table in third-normal-form relational design** - except dimensional modelers usually avoid those. Here the bridge is explicitly permitted because the business reality is genuinely many-to-many and any flattening would lose information or corrupt aggregations.

⚠️ WARNING

Bridges can cause double-counting if not used carefully. If you JOIN from a fact to a bridge without weighting, each fact row multiplies by the number of bridge rows. A transaction with 3 diagnoses joined to the bridge returns 3 rows.

Always aggregate at the right granularity or use weight factors to prevent inflated totals.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Adding multiple FK columns to the fact table: `diagnosis1_key`, `diagnosis2_key`, `diagnosis3_key`. This limits the maximum number of diagnoses (what if a patient has 10?), creates nullable columns for lower counts, and makes filtering by diagnosis unwieldy.

Correct Way: Bridge table with one row per encounter × diagnosis combination. No maximum limit. Clean joins. Filterable by any diagnosis attribute.

EXAM TIP CARD

Interview classic: "How do you model a fact where one transaction can be attributed to multiple cost centers with different percentages?" Winning answer: *"Bridge table. Grain: `transactionkey` × `costcenterkey` × `weightfactor`.

The `weightfactor` represents the percentage allocation (must sum to 1.0 per transaction). In the query: JOIN fact to bridge on `transactionkey`, JOIN bridge to `dimcostcenter` on `costcenterkey`.

Aggregate with `SUM(amount weight_factor)` to get correctly weighted cost-center totals that foot to the original transaction total."

🔥 EXERCISE

A university awards grants that can be shared across multiple research projects. Each project gets a different percentage of the grant.

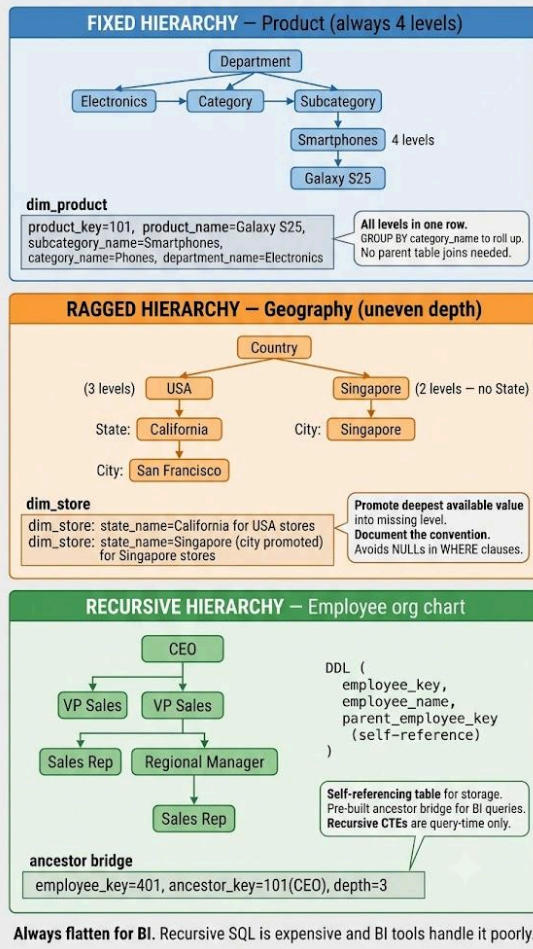
Design: (a) the grant fact table, (b) the `dim_project` dimension, (c) the bridge table with weight factor. Write the SQL to find total grant dollars allocated to each department, weighted correctly.

💡 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Design a bridge table for a retail loyalty program where each transaction can be split across multiple promotional campaigns (each campaign gets credit for a portion of the sale). Include: fact table DDL, `dim_campaign` DDL, bridge DDL with `weight_factor`, the SQL to compute campaign-attributed revenue, and the correctness check query that verifies weighted totals equal unweighted fact totals."

HIERARCHIES IN DIMENSIONS — FIXED, RAGGED, AND RECURSIVE

Three Hierarchy Types — Fixed, Ragged, and Recursive.



Three Hierarchy Types - Fixed, Ragged, and Recursive

sql – Fixed hierarchy DDL and recursive hierarchy flattening with CTE

```
-- =====
-- FIXED HIERARCHY: product with four flattened levels
-- =====

CREATE TABLE dim_product (
    product_key      INT PRIMARY KEY,
    product_code     VARCHAR(20) NOT NULL,
    product_name     VARCHAR(200) NOT NULL,
    subcategory_key  INT,
    subcategory_name VARCHAR(100),
    category_key     INT,
    category_name    VARCHAR(100),
    department_key   INT,
    department_name  VARCHAR(100),
    brand            VARCHAR(100),
    is_active        TINYINT NOT NULL DEFAULT 1
);
```

```
-- Roll up to category – no extra joins
SELECT category_name, SUM(f.net_revenue) AS category_revenue
FROM fact_sales_transaction f
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY category_name
ORDER BY category_revenue DESC;

-- =====
-- RECURSIVE HIERARCHY: employee org chart
-- =====

CREATE TABLE dim_employee (
    employee_key      INT PRIMARY KEY,
    employee_name      VARCHAR(200),
    job_title          VARCHAR(100),
    parent_employee_key INT REFERENCES dim_employee(employee_key) -- self-
reference
);

-- Recursive CTE: all subordinates of manager 101
WITH RECURSIVE org_tree AS (
    -- Anchor: the manager themselves
    SELECT employee_key, employee_name, parent_employee_key, 0 AS depth
    FROM dim_employee
    WHERE employee_key = 101

    UNION ALL

    -- Recursive: add each employee whose parent is already in the result
    SELECT e.employee_key, e.employee_name, e.parent_employee_key, t.depth +
1
    FROM dim_employee e
    JOIN org_tree t ON e.parent_employee_key = t.employee_key
)
SELECT * FROM org_tree ORDER BY depth, employee_name;

-- =====
-- PRE-BUILT ANCESTOR BRIDGE: rebuild nightly for BI use
-- =====

CREATE TABLE bridge_employee_hierarchy (
    employee_key INT NOT NULL,
    ancestor_key INT NOT NULL,
    depth        INT NOT NULL, -- 1 = direct manager, 2 = skip-level, etc.
    PRIMARY KEY (employee_key, ancestor_key)
);

-- Populate from the recursive CTE above, inserting all ancestor pairs
-- BI query: total sales for all reps under manager 101
```

```
SELECT SUM(f.net_revenue)
FROM fact_sales_transaction f
JOIN bridge_employee_hierarchy h
    ON f.employee_key = h.employee_key
    AND h.ancestor_key = 101;
```

Student

How do I handle hierarchies? My product dimension has four levels - product, subcategory, category, department - and my org chart has uneven depth.

Claude • AI Assistant

Hierarchies are the mechanism that enables **drill-down and drill-up** in BI tools. Kimball's canonical approach to hierarchies is to flatten them into the dimension row rather than use separate tables or recursive SQL - because flat columns are what columnar databases and BI tools handle fastest.

CONCEPT CARD

Fixed-depth hierarchy - the standard case:

1. **Definition** - every leaf node is at the same depth from the root. Products always have exactly: product → subcategory → category → department.
2. **Implementation** - flatten all levels into a single dimension row. One row per leaf (product). Parent attributes stored redundantly on every child.
3. **Columns** - `product_key`, `product_name`, `subcategory_key`, `subcategory_name`, `category_key`, `category_name`, `department_key`, `department_name`.
4. **Query benefit** - GROUP BY `category_name` to roll up to category. GROUP BY `department_name` to roll up to department. No joins to parent tables needed.
5. **Redundancy is intentional** - `category_name` repeated on every product row in that category. This is correct in dimensional modeling. The query performance benefit far outweighs the storage cost.

CONCEPT CARD

Ragged hierarchy - uneven depth:

1. **Definition** - not all leaf nodes are at the same depth. A geography hierarchy may have Country → State → City → Zip for the US, but Country → City for Singapore (no State level).

2. **Problem** - you cannot simply use fixed-depth columns; missing levels leave NULLs in unintuitive places.
3. **Implementation option 1** - fixed columns with NULLs for missing levels. Simple but awkward for queries.
4. **Implementation option 2** - add an `is_leaf TINYINT` flag. Leaf nodes get their own row; intermediate nodes also get rows. A bridge-style parent-child table connects them.
5. **Implementation option 3 (preferred)** - normalize the hierarchy to the deepest level present for each country. Repeat the deepest available value upward into missing levels: Singapore gets `city_name` in the `state_name` column too.

CONCEPT CARD**Recursive (parent-child) hierarchy** - unknown depth:

1. **Definition** - each row has a `parent_key` pointing to another row in the same table. An employee's manager is also an employee. A product category's parent category is also a category.
2. **DDL** - `dim_employee(employee_key, employee_name, parent_employee_key)` - self-referencing FK.
3. **Problem** - SQL queries on parent-child trees require recursive CTEs. Analytical tools handle this poorly. Drill-down in BI tools is difficult.
4. **Flatten for BI** - for reporting use, pre-compute a flattened hierarchy table listing every ancestor for every node: `hier_employee(employee_key, ancestor_key, depth)` . Rebuild nightly. BI tools then do simple joins.
5. **Depth column** - `depth INT` tells you how many levels up the ancestor is. `depth = 1` = direct manager. `depth = 3` = skip-level-skip.

EXAMPLE CARD

Retail product fixed hierarchy - full DDL:

sql – Code Example

```
CREATE TABLE dim_product (
    product_key      INT PRIMARY KEY,
    product_code     VARCHAR(20),
    product_name     VARCHAR(200),
    -- Hierarchy: flattened into dimension row
    subcategory_key  INT,
    subcategory_name VARCHAR(100),
    category_key     INT,
    category_name    VARCHAR(100),
    department_key   INT,
    department_name  VARCHAR(100),
    -- Additional product attributes
```

```

brand          VARCHAR(100),
unit_of_measure VARCHAR(20),
is_active      TINYINT DEFAULT 1
);

```

🔗 ANALOGY

A fixed hierarchy in a flat dimension row is like **a filing system where every file folder shows the full path** - 'Department: Electronics > Category: Phones > Subcategory: Smartphones > Product: Model X.' You never have to open parent folders to know where you are in the hierarchy. The full path is on the label.

⚠️ WARNING

Do not store hierarchy levels as separate dimension tables (department table, category table, subcategory table) and join them at query time. This is dimensional snowflaking and it creates complex multi-join queries, defeats columnar compression, and confuses BI tools that expect flat dimension tables.

Flatten the hierarchy.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Storing only `subcategory_key` in the product dimension and joining to a separate `dim_subcategory` table that joins to `dim_category` which joins to `dim_department`. Three extra joins per query, poor BI tool compatibility, and incorrect grain if the subcategory-category relationship changes.

Correct Way: All four levels flattened into `dim_product`. `GROUP BY category_name` works directly without any additional joins.

EXAM TIP CARD

Interview classic: "How do you handle a geographic hierarchy where some countries have states and some don't?" Winning answer: "Ragged hierarchy. Two options: (1) fixed-width columns with NULL in the state column for countries without states - simple but awkward. (2) Bridge to a flattened ancestor table - each geography node lists all its ancestors with depth. For most BI workloads: use fixed columns and promote the most granular level available into the missing level column (*cityname into statename* for small countries), making NULL-handling explicit.

Document the convention."

🔥 EXERCISE

Build a fixed-depth hierarchy for a retail geography: `store` → `city` → `state` → `region` → `country`. Write the `dim_store` DDL with all levels flattened. Write a query that shows total revenue by region and by state, using only `dim_store` (no additional joins).

🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: *""Design a recursive employee hierarchy for an org chart in SQL. Include: the self-referencing dimemployee table, a recursive CTE to find all employees under a given manager, a flattened hierarchy bridge table (employeekey, ancestor_key, depth), and the SQL to rebuild the bridge from the self-referencing table.*

Explain why the flattened bridge is preferable for BI reporting.""

DATE AND TIME DIMENSIONS — THE MOST REUSED DIMENSION

Claude • AI Assistant

Date Dimension — Pre-Built Calendar vs SQL Date Functions

SQL DATE FUNCTIONS — what they can answer	DATE DIMENSION — what it answers simply
✓ SELECT YEAR(sale_date) — calendar year only.	✓ WHERE d.fiscal_quarter = 3 — pre-computed per company calendar.
✓ SELECT MONTH(sale_date) — calendar month.	✓ WHERE d.is_holiday = 1 — holiday flag from business data.
✗ SELECT fiscal_quarter — not computable from date math alone.	✓ WHERE d.relative_period_month = -1 — last month, automatically updated.
✗ WHERE is_holiday = 1 — no SQL function for business holidays.	✓ d.week_of_fiscal_year for same-week comparisons.
✗ WHERE relative_period = -1 (last month) — requires complex DATEADD logic per query.	✓ d.fiscal_year, d.fiscal_month for any fiscal calendar.
✗ Same-week-last-year comparison — messy 52/53-week year edge cases.	✓ d.is_business_day, d.shift_name for operational analytics.

dim_date size: 10 years = 3,653 rows. Trivially small. Cost: one pre-build query.

Benefit: every report team uses the same calendar definitions forever.

Date dimension is the highest ROI dimensional investment in any warehouse — built once, used in every fact table.

Date Dimension - Pre-Built Calendar vs SQL Date Functions

```
sql – Date dimension DDL and fiscal-calendar population

-- =====
-- COMPLETE dim_date DDL
-- =====

CREATE TABLE dim_date (
    date_key          INT          PRIMARY KEY,      -- YYYYMMDD
    calendar_date     DATE          NOT NULL UNIQUE,
    day_name          VARCHAR(10) NOT NULL,         -- 'Monday'
    day_of_month      TINYINT      NOT NULL,
    day_of_week_num   TINYINT      NOT NULL,         -- 1=Sunday, 7=Saturday
```

```

is_weekend          TINYINT    NOT NULL,
week_of_year        TINYINT    NOT NULL,
month_number        TINYINT    NOT NULL,
month_name          VARCHAR(10) NOT NULL,
quarter_number      TINYINT    NOT NULL,      -- calendar quarter
year_number         SMALLINT   NOT NULL,
-- Fiscal calendar (April 1 fiscal year start)
fiscal_year         SMALLINT   NOT NULL,
fiscal_quarter      TINYINT    NOT NULL,
fiscal_month        TINYINT    NOT NULL,      -- 1=April, 12=March
fiscal_week         TINYINT    NOT NULL,
-- Business attributes
is_holiday          TINYINT    NOT NULL DEFAULT 0,
holiday_name        VARCHAR(100),
is_business_day     TINYINT    NOT NULL DEFAULT 1,
-- Relative periods (updated by ETL on each load)
relative_period_month INT      NOT NULL DEFAULT 0,  -- 0=current,
-1=last
is_last_7_days      TINYINT    NOT NULL DEFAULT 0,
is_last_30_days     TINYINT    NOT NULL DEFAULT 0,
is_last_365_days    TINYINT    NOT NULL DEFAULT 0
);

-- =====
-- FISCAL YEAR LOGIC: April 1 start
-- For dates Apr 1 - Mar 31, fiscal_year = calendar_year + 1
-- e.g., Apr 1 2024 - Mar 31 2025 → fiscal_year 2025
-- =====
UPDATE dim_date SET
    fiscal_year = CASE WHEN month_number >= 4
                      THEN year_number + 1
                      ELSE year_number END,
    fiscal_month = CASE WHEN month_number >= 4
                      THEN month_number - 3
                      ELSE month_number + 9 END,  -- April=1, March=12
    fiscal_quarter = CASE
                      WHEN month_number IN (4,5,6) THEN 1
                      WHEN month_number IN (7,8,9) THEN 2
                      WHEN month_number IN (10,11,12) THEN 3
                      ELSE 4
    END;

-- =====
-- RELATIVE PERIOD: update monthly (run in ETL job)
-- =====
UPDATE dim_date SET

```

```

relative_period_month = (
    (year_number * 12 + month_number)
    - (YEAR(CURRENT_DATE) * 12 + MONTH(CURRENT_DATE))
),
is_last_30_days = CASE WHEN calendar_date >= CURRENT_DATE - INTERVAL '30
days'
                        AND calendar_date <= CURRENT_DATE THEN 1 ELSE 0
END;

```

Student

Why do I need a pre-built date dimension? Can't I just use `YEAR(sale_date)` and `MONTH(sale_date)` in my queries?

Claude • AI Assistant

You can - and most junior teams do. Then they get their first request for 'sales in our fiscal Q3' and discover their fiscal year ends in March, not December.

Or they need 'sales excluding public holidays.' Or 'week-over-week same-day comparison.' SQL date functions compute calendar dates. Business calendars - fiscal years, holiday schedules, relative periods - are data, not functions.

The date dimension is the most leveraged investment in any warehouse because every single fact table uses it.

CONCEPT CARD

Why a pre-built date dimension beats SQL date functions:

1. **Fiscal calendars** - `YEAR(date)` gives the calendar year. `fiscal_year` in the date dimension gives your company's fiscal year - which may start in April, July, or October.
2. **Holiday flags** - `is_holiday` is a business decision, not a mathematical property of a date. There is no SQL function for 'Diwali 2025.'
3. **Query performance** - filtering `WHERE d.fiscal_quarter = 3` on a pre-built integer column is faster than `WHERE DATEPART(QUARTER, date) = 3` on a raw date column, especially at billions of rows.
4. **Relative periods** - `WHERE d.relative_period = -1` gives last month automatically. No complex `DATEADD` logic in every report.
5. **Consistency** - every report team uses the same definition of 'Q3' and 'week number.' No more conflicting calendar interpretations across departments.

CONCEPT CARD

Essential columns for a production date dimension:

1. **`date\_key INT`** - surrogate, typically formatted as YYYYMMDD (e.g., 20241105). Enables fast range filters.
2. **Calendar attributes** - `calendar_date DATE` , `day_name VARCHAR` , `day_of_month` , `month_name` , `month_number` , `quarter_number` , `year_number` , `is_weekend` , `week_of_year` .
3. **Fiscal attributes** - `fiscal_year` , `fiscal_quarter` , `fiscal_month` , `fiscal_week` - follow your company's fiscal calendar.
4. **Holiday attributes** - `is_holiday TINYINT` , `holiday_name VARCHAR` (NULL when not a holiday), `is_business_day TINYINT` .
5. **Relative period** - `relative_period_month INT` - 0 for the current month at load time, -1 for last month, -12 for same month last year. Updated monthly as the warehouse loads.
6. **Last-N-day flags** - `is_last_7_days` , `is_last_30_days` , `is_last_365_days` - updated daily. Enables simple one-column filters in BI dashboards.

CONCEPT CARD

Time dimension (separate from date):

1. For sub-daily fact tables (clickstream, IoT sensor data, call center), add a separate `dim_time` with one row per second or minute.
2. **Columns** - `time_key INT` (HHMMSS), `hour INT` , `minute INT` , `am_pm VARCHAR` , `business_hour_flag TINYINT` , `shift_name VARCHAR` (morning/evening/night).
3. **Never combine date + time into one big dimension** - a combined table has 86,400 rows per day × 365 days × 10 years = 315 million rows. Keep them separate; join both keys in the fact.

🔗 ANALOGY

A date dimension is like the **master calendar on a factory floor** - the physical wall calendar that marks holidays, shift changes, fiscal quarter endings, and production deadlines. Every team in the factory refers to the same calendar.

If it's not on the wall calendar, it doesn't count as a official scheduling constraint. SQL `DATEPART()` is like each worker doing calendar math in their head - everyone might get a different answer for 'is this a holiday?'

⚠️ WARNING

Populate at least 10 years backward and 5 years forward from your oldest historical data. Fact tables that join on `date_key` will silently drop rows where the date key has no matching row in `dim_date` (referential integrity).

A date 10 years in the future (a delivery date estimate, a contract expiry) falling outside the dimension range produces a NULL join - incorrect aggregations with no error message.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Using a date dimension with `date_key` as the actual `DATE` type (e.g., `date_key DATE = '2024-11-05'`). Foreign keys in facts are then DATE columns rather than INT - slower joins, larger storage in high-volume facts, and vendor-specific date handling differences.

Correct Way: `date_key INT = 20241105` (YYYYMMDD integer). All foreign keys in fact tables are INT. Joins on INT are the fastest possible key join type.

EXAM TIP CARD

Interview classic: "Why use a date dimension instead of extracting year/month/quarter from a date column at query time?" Winning answer: "Three reasons. First, fiscal calendars: YEAR() gives the calendar year; a date dimension gives the company's fiscal year - not computable from the date alone.

Second, business attributes: holiday flags, business-day indicators, and special event markers are business data, not mathematical properties. Third, performance: filtering on pre-computed integer columns is faster than applying date functions to billions of rows on every query."

🔥 EXERCISE

Your company's fiscal year runs from April 1 to March 31. Calendar year 2024 covers two fiscal years: FY2024 (Apr 2023–Mar 2024) and FY2025 (Apr 2024–Mar 2025).

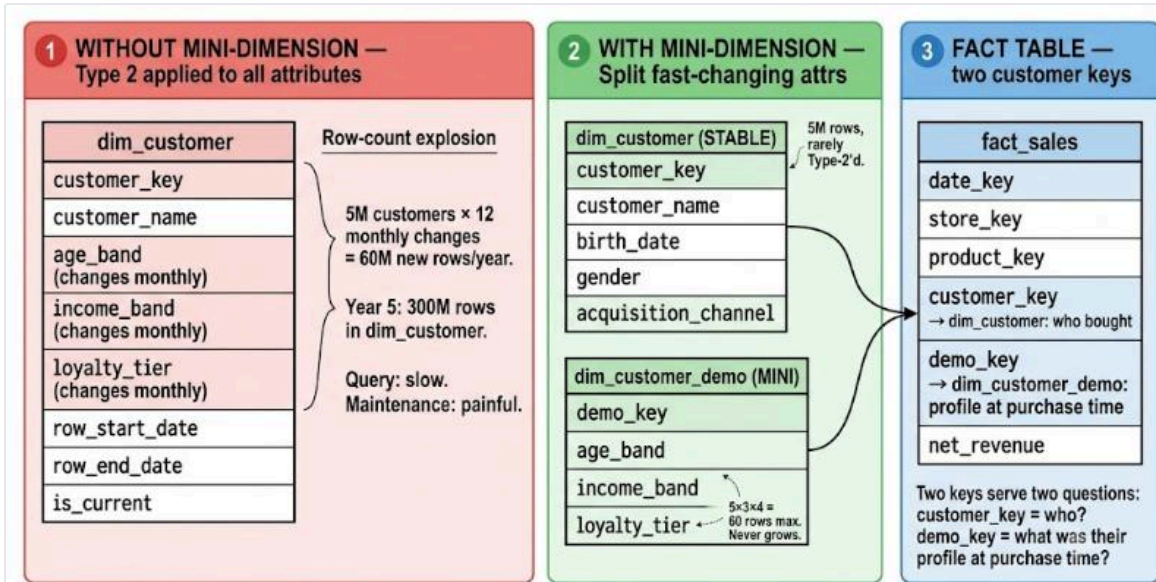
(a) Write the UPDATE statement that sets `fiscal_year` correctly for dates in calendar year 2024. (b) Write the query that compares FY2024 Q1 revenue to FY2025 Q1 revenue using the date dimension.

🧙 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Generate a complete *dimdate* table for fiscal year April-to-March with 20 years of data (2015 to 2035). Include: *datekey* as YYYYMMDD integer, all calendar attributes, fiscal year/quarter/month computed from the April start, *isholiday* placeholder, *isweekend*, *isbusinessday*, *relativeperiodmonth*, and *islast30_days* flag.

Show the SQL to generate the rows using a recursive CTE or series generator."

MINI-DIMENSIONS — SPLITTING RAPID-CHANGE ATTRIBUTES



Mini-Dimension — Splitting Rapid-Change Attributes to Prevent Type 2 Row Explosion.

Mini-Dimension - Splitting Rapid-Change Attributes to Prevent Type 2 Row Explosion

sql — Mini-dimension DDL, population, and ETL lookup

```
-- =====
-- MAIN CUSTOMER DIMENSION: stable attributes only
-- =====
CREATE TABLE dim_customer (
    customer_key      INT PRIMARY KEY,
    customer_id       VARCHAR(20) NOT NULL, -- natural key
    customer_name     VARCHAR(200) NOT NULL,
    birth_date        DATE,
    gender            CHAR(1),
    acquisition_channel VARCHAR(50),
    first_purchase_date DATE
    -- NO age_band, income_band, loyalty_tier here
);

-- =====
-- MINI-DIMENSION: rapidly changing profile attributes
-- Max rows = 5 age bands × 3 income bands × 4 tiers = 60
-- =====
CREATE TABLE dim_customer_demo (
    demo_key          INT PRIMARY KEY,
    age_band          VARCHAR(10) NOT NULL, -- '18-24', '25-34', '35-44', '45-54', '55+'
    income_band       VARCHAR(10) NOT NULL, -- 'Low', 'Medium', 'High'
    loyalty_tier      VARCHAR(10) NOT NULL --
```

```

'Bronze','Silver','Gold','Platinum'
);

-- Populate with Cartesian product of all combinations
INSERT INTO dim_customer_demo (demo_key, age_band, income_band, loyalty_tier)
SELECT
    ROW_NUMBER() OVER (ORDER BY a.age_band, i.income_band, l.loyalty_tier),
    a.age_band, i.income_band, l.loyalty_tier
FROM
    (VALUES ('18-24'),('25-34'),('35-44'),('45-54'),('55+')) a(age_band)
CROSS JOIN
    (VALUES ('Low'),('Medium'),('High'))
i(income_band)
CROSS JOIN
    (VALUES ('Bronze'),('Silver'),('Gold'),('Platinum'))
l(loyalty_tier);

-- =====
-- FACT TABLE: two customer-related keys
-- =====
CREATE TABLE fact_sales_transaction (
    date_key      INT NOT NULL,
    store_key     INT NOT NULL,
    product_key   INT NOT NULL,
    customer_key  INT NOT NULL,    -- FK → dim_customer (identity)
    demo_key      INT NOT NULL,    -- FK → dim_customer_demo (profile at
purchase)
    net_revenue   DECIMAL(12,2),
    quantity_sold INT
);

-- =====
-- ETL LOOKUP: resolve demo_key at load time
-- =====
SELECT demo_key
FROM   dim_customer_demo
WHERE  age_band      = :customer_current_age_band
      AND income_band = :customer_current_income_band
      AND loyalty_tier = :customer_current_loyalty_tier;
-- Store returned demo_key in the fact row

-- =====
-- ANALYSIS: revenue by loyalty tier AT TIME OF PURCHASE
-- =====
SELECT
    d.loyalty_tier,
```

```
SUM(f.net_revenue) AS revenue_at_purchase_tier
FROM fact_sales_transaction f
JOIN dim_customer_demo d ON f.demo_key = d.demo_key
GROUP BY d.loyalty_tier
ORDER BY revenue_at_purchase_tier DESC;
```

Student

My customer dimension has 5 million rows. Some attributes like age band, income bracket, and loyalty tier change every month. If I use Type 2 SCD, I'll have 60 million rows in a year. What do I do?

Claude • AI Assistant

You use a **mini-dimension** - one of Kimball's most elegant solutions. The rapidly-changing attributes are split out of the main dimension into a small, separate dimension with its own surrogate key.

The fact table carries two customer-related keys: the original **customer_key** (stable, points to the main customer profile) and a new **demo_key** (points to the mini-dimension row that was current at the time of the transaction).

CONCEPT CARD

Why mini-dimensions exist:

1. **The Type 2 problem** - if **dim_customer** has 5 million customers and 10 attributes change monthly, Type 2 generates $5M \times 12 = 60M$ new rows per year. Query performance collapses.
2. **The mini-dimension trade-off** - move the fast-changing attributes to a tiny dimension. **dim_customer_demo** has at most **num_bands_of_each_attribute** combinations - typically dozens to a few thousand rows.
3. **The fact gets a second key** - **fact_sales(customer_key, demo_key, ...)**. **customer_key** identifies WHO bought. **demo_key** identifies the DEMOGRAPHIC PROFILE of that customer at the time of purchase.
4. **The main dimension stays stable** - **dim_customer** now carries only slowly-changing or permanent attributes: name, date of birth, gender, acquisition channel. It rarely generates Type 2 rows.

CONCEPT CARD

Mini-dimension design:

1. **Identify the rapid-change attributes** - *ageband, incomeband, loyaltytier, creditscorerange, churnrisk_segment*.
2. **Define the value domains** - *ageband*: '18-24', '25-34', '35-44', '45-54', '55+'.
incomeband: 'Low', 'Medium', 'High'. *loyalty_tier*: 'Bronze', 'Silver', 'Gold', 'Platinum'.
3. **Build the mini-dimension** - all combinations of the defined values: $5 \times 3 \times 4 = 60$ rows. Very small.
4. **Populate at transaction time** - when a sale is loaded, look up the customer's current demographic profile; find the matching mini-dimension row; store its `demo_key` in the fact.
5. **Update the main dimension** - just update the current value in place (Type 1) on `dim_customer`. The fact table captures the historical demographic state via `demo_key`.

CONCEPT CARD

Querying with a mini-dimension:

1. **Current profile query** - JOIN `fact` to `dim_customer_demo` via `fact.demo_key`.
Attributes are as of transaction time.
2. **Current-state query** - JOIN `fact` to `dim_customer` via `fact.customer_key`.
Attributes are current (Type 1 updated).
3. **Cohort analysis** - 'Revenue from customers who were in Gold tier at time of purchase vs now': two joins, one per key.
4. **The problem of connecting main and mini** - there is no permanent FK between `dim_customer` and `dim_customer_demo`.

They connect only through the fact. This is a known limitation - addressed with an 'outrigger' technique (a bridge fact with daily snapshots of customer-to-demo mapping).

🔗 ANALOGY

A mini-dimension is like **a separate wardrobe tag on a clothing item**. The garment itself (main customer dimension) carries permanent attributes: material, manufacturer, style.

The wardrobe tag (mini-dimension) carries today's state: current price tier, current condition, current location in the store. When you sell the garment (transaction), you record both the garment ID and the current wardrobe tag.

Tomorrow the tag may change, but today's sale forever remembers which tag was attached.

⚠️ WARNING

Mini-dimensions solve the row explosion problem but introduce a **join gap**: you cannot query the current attribute values alongside the historical fact data in one simple join. You need either (a) the outrigger bridge, or (b) acceptance that fact-level queries show demographic state at-purchase-time, not current state.

Document this behavior explicitly for BI consumers.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Applying Type 2 to ALL attributes in `dim_customer`, including rapidly-changing ones like `credit_score_range`. 5M customers × 12 updates per year = 60M rows per year → 600M rows in decade → warehouse collapses under the maintenance load.

Correct Way: Type 2 only on slowly-changing attributes (name, address). Mini-dimension for rapidly-changing attributes (age band, loyalty tier, income bracket). Two different keys in the fact table serve two different temporal purposes.

EXAM TIP CARD

Interview classic: "A customer dimension has 10 million rows and several attributes that change monthly. How do you handle this?" Winning answer: "Mini-dimension."

Split the rapidly-changing attributes (income band, loyalty tier, churn segment) into a separate `dimcustomerdemo` with one row per valid combination - typically dozens to a few thousand rows. Add `demokey` to the fact table alongside `customerkey`.

Load the current demo profile at transaction time. The main `dim_customer` carries only stable or slowly-changing attributes and is managed with Type 1 or Type 2 normally.

Row count is controlled; historical state is preserved via `demo_key`."

🔥 EXERCISE

Design a mini-dimension for a telecom company's subscriber dimension. Attributes that change monthly: `data_plan_tier` (Basic/Standard/Premium), `payment_behavior` (OnTime/Late/Default), `usage_band` (Low/Medium/High/Heavy).

(a) How many rows in the fully populated mini-dimension? (b) Write the DDL.

(c) Write the ETL lookup SQL to find the correct `demo_key` for a new fact row.

🧙 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Design a complete mini-dimension solution for a retail customer dimension with 8 million customers. Rapidly-changing attributes: `loyaltytier` (4 values), `spendband` (5 values), `churnrisk` (3 values), `preferredchannel` (4 values).

Include: `dimcustomerdemo` DDL, the Cartesian product population SQL, the fact table DDL with both `customerkey` and `demokey`, the ETL lookup query, and a sample analysis query comparing revenue by loyalty tier at time of purchase vs current loyalty tier."

CHAPTER REVIEW & SUMMARY

Claude • AI Assistant

CHAPTER SUMMARY

You now have five advanced dimension patterns that handle the scenarios a simple flat lookup table cannot.

- **ROLE-PLAYING DIMENSIONS:** One physical table, multiple logical meanings. Create one database view per role (`CREATE VIEW dimshipdate AS SELECT * FROM dim_date`). Fact table holds one FK column per role. BI tools see each view as an independent dimension. Never copy the physical table - one maintenance point is the design goal. Query-time aliasing is the alternative for ad-hoc SQL.
- **BRIDGE TABLES:** Resolve many-to-many between fact and dimension. Bridge holds grain key + dimension key + optional weight_factor. Weights must sum to 1.0 per fact grain row. Query: `fact JOIN bridge JOIN dimension`. Aggregation uses `SUM(measure × weight)` for allocated measures. Correctness check: weighted totals must equal unweighted fact totals. Alternative to repeating rows (which inflates aggregations) or multi-value columns (which break relational joins).
- **HIERARCHIES:** Fixed (all leaves at same depth) - flatten all levels into one dimension row, `GROUP BY` any level column directly. Ragged (uneven depth) - promote deepest available value into missing level columns; document the convention. Recursive (parent-child self-reference) - pre-build an ancestor bridge table nightly for BI use; recursive CTEs are for ad-hoc only. Never snowflake fixed hierarchies into separate parent tables.
- **DATE DIMENSION:** Pre-built, not computed. One row per calendar day. YYYYMMDD integer key. Fiscal year, quarter, month for company-specific calendar. *isholiday*, *isbusinessday*, *holidayname* for business rules. *relativeperiodmonth* for rolling-window filters. *islast30days*, *islast7days* updated daily. Populate 10 years back and 5 years forward. Keep date and time as separate dimensions for sub-daily facts.
- **MINI-DIMENSIONS:** Split rapidly-changing attributes from a large SCD to prevent Type 2 row explosion. Mini-dimension has one row per valid combination of rapid-change attributes - typically dozens to a few thousand rows. Fact table gains a second dimension key (demokey) *alongside the main key (customerkey)*. Main dimension stays stable. Historical demographic state is preserved via `demo_key` at transaction load time. Known limitation: no direct join between main and mini outside the fact table.

Claude • AI Assistant

With advanced dimension techniques in hand, Chapter 3 scales the view to the enterprise: how data marts are scoped and governed, how the Kimball Bus Matrix blueprints an entire warehouse incrementally while guaranteeing cross-subject integration, and two complete worked examples - retail and e-commerce - showing these patterns in production-scale domain models.